

Introducción al Modelo Relacional

El modelo relacional es el marco en el cuál se basan los DBMS relacionales. Consiste en un modelo matemático-lógico basado en la lógica de predicados y la teoría de conjuntos. Fue originalmente escrito por Edgar Codd hacia 1969 y publicado originalmente por IBM y fue presentado cómo una evolución de los modelos jerárquicos y de red.

Cómo todo marco teórico es conveniente hacer una introducción a su terminología y sus conceptos:

Tupla:

Matemáticamente consiste en una lista finita y ordenada de elementos.

Lo podemos pensar dentro del modelo relacional cómo un par ordenado de orden n (siendo n la cardinalidad - o cantidad de campos - de la relación a la que la tupla pertenece). Normalmente (por sugerencia de Codd) se utiliza el nombre del atributo definido en lugar del orden en la tupla para acceder a una dimensión de la misma y prescindir del orden, ya que no tiene sentido pensar en función de conjuntos ordenados, sino cómo correspondientes al orden arbitrario dado en una definición.

Relación

Es el principal de los conceptos dentro del modelo relacional. Dicha estructura es fundamental para representar datos. Estrictamente podríamos definir a una relación cómo un conjunto de tuplas. El equivalente informal en el modelo de entidad relación para la relación es la "tabla". Conviene introducir tempranamente esta analogía, ya que puede ayudar al lector a comprender el modelo. Asimismo la "tabla" se dice que es la instancia o el ejemplar de relación y que la definición o cabecera de la misma es conocido como el esquema de relación. Para describir un esquema de relación, se enumeran (el orden es importante) los campos o atributos que describirá la relación definida. Por ejemplo una relación "empleados" puede ser descripta por el siguiente esquema de relación:

Empleados (legajo : integer, nombre : string, fecha_ingreso : date)

En dónde declaramos la existencia de una relación Empleados compuesta de un legajo (un número entero), un nombre (una cadena de caracteres) y una fecha de ingreso (una fecha).

Un ejemplar de relación, o tabla (informalmente) consiste en un conjunto de **tuplas**, que informalmente son conocidas como registros o filas. Por ejemplo, utilizando el esquema de relación definido arriba, podemos diagramar fácilmente un ejemplo de tabla o ejemplar de relación:

legajo	nombre	fecha_ingreso
131250	Carlos Padilla	15/03/1978
131251	Ramiro Ochoa	17/10/2004
131252	Daniel Ramirez	10/04/2007
131253	Agustín Tarco	25/05/1999
131254	Javier Salazar	15/03/2007

Por tanto, podemos decir, que la relación Empleados, posee un esquema definido de la siguiente manera:

Empleados (legajo:integer, nombre:string, fecha_ingreso:date)

Y que dicha relación está definida por las tuplas que contiene, podemos definir la relación como un conjunto de tuplas de la siguiente manera:

Empleados = { (131250,' Carlos Padilla', 15/03/1978) ;
 (131251,' Ramiro Ochoa, 15/03/1978) ;
 (131252,' Daniel Ramirez', 15/03/1978) ;
 (131253,' Agustín Tarco, 15/03/1978) ;
 (131254,' Javier Salazar', 15/03/1978) }

Conviene destacar ciertas propiedades que poseen las relaciones, que se desprenden de su definición.

Definición formal de Relación:

Declarado un multiconjunto de Dominios o Tipos **T**, por ejemplo

$T = \{integer;string;integer;...;double\}$

Se dice que **R(T)** es una relación sobre el conjunto de tipos **T** si existen:

- Un esquema de Relación o encabezado
- Un ejemplar de relación o cuerpo que cumplan con las siguientes condiciones:
 - El encabezado o esquema de relación es un conjunto de atributos **A(R)**, donde atributo generalmente se expresa en la forma de "*nombreAtributo:dominio*" en donde no puede haber nombres de atributo duplicados y los dominios son los declarados en **T**.
 - El cuerpo o ejemplar de relación es un conjunto de tuplas **P(R)**, donde para todas las tuplas de **P(R)** se componen de componentes en la forma "*nombreAtributo:valor*" que se corresponden con **A(R)**. Cada tupla en particular contiene exactamente un valor por cada atributo definido en el esquema de relación.

Otra definición de ejemplar de Relación se puede expresar como un subconjunto del producto cartesiano entre todos los conjuntos de dominios **T** que lo definen en su esquema de relación, por tanto la relación **R** está incluida en el conjunto definido por:

$$\text{dominio}(\text{atributo1}) \times \text{dominio}(\text{atributo2}) \times \dots \times \text{dominio}(\text{atributo } n)$$

Se define **cardinalidad de la relación R** a la cantidad de elementos del cuerpo o ejemplar de relación **P(R)**.

Se define **grado de la relación R** a la cantidad de elementos del encabezado o esquema de relación **A(R)**.

Desde el punto de vista de la lógica de predicados, se puede afirmar a ciencia cierta que un esquema de relación puede ser considerado un predicado en sí. También las tuplas pertenecientes a un ejemplar de relación de una relación **R** pueden ser consideradas proposiciones verdaderas, que se obtienen a partir del esquema de relación vía la sustitución de valores (del dominio correspondiente) por el parámetro de posición del predicado del encabezado. También podemos afirmar que dada una tupla posible válida para una relación **R**, y no pertenece al ejemplar de relación **P(R)**, estaríamos ante una proposición con valor de verdad falso, o más claro aún, que el ejemplar de relación **P(R)** contiene sólo tuplas que correspondan a proposiciones con valor de verdad verdadero.

Dado que la teoría de conjuntos no asume la existencia de una relación de orden entre los componentes de un conjunto, los ejemplares de relación (vistos como conjuntos de tuplas) no garantizan ningún orden específico de presentación (ni de almacenamiento) de sus tuplas.

Con respecto al orden de los atributos dentro de una tupla, no consiste en una necesidad del modelo, se suele representarlos con el mismo orden que se asignan los atributos y dominios en la definición del esquema de relación, pero tranquilamente se podría

pensar las tuplas cómo un conjunto de pares ordenados ("atributo","valor"), con lo cual el orden es totalmente innecesario.

Dominio

Consiste en un tipo de dato. Es el conjunto de partida para cualquier instancia de un atributo en particular dentro de una tupla. Es también el conjunto de valores definido para un tipo de datos, es decir todos los valores posibles de un tipo particular de datos. Los tipos de datos pueden ser "Simples" (o definidos por el sistema) o complejos (o definidos por el usuario). Generalmente utilizaremos tipos de datos simples, ya que los complejos carecen de popularidad en el ambiente del desarrollo de aplicaciones por poseer poca portabilidad e implicar un problema a la hora de mapear un valor con dicho dominio.

Cabe destacar que todo valor existente dentro del modelo relacional tiene un tipo, y un dominio al cual pertenece. Por ejemplo, puedo decir que dentro del modelo puede existir el valor "4", pero tendré que aclarar si pertenece a los integer, los double, etc. Conviene pensar cualquier valor cómo instancia concreta de un tipo.

A su vez, cómo en los lenguajes de programación, todo tipo puede ser escalar o no escalar. Un tipo escalar es de naturaleza atómica y no tiene partes divisibles. Un tipo no escalar es aquel que se encuentra definido explícitamente para tener partes.

El modelo relacional soporta tanto la definición de tipos abstractos de datos (permitiéndonos el abuso de notación) y la declaración de operadores en el marco de la lógica de predicados.

Una vez presentados estos conceptos, podemos ensayar la definición de Base de datos relacional cómo un conjunto de relaciones (encabezados o esquemas de relación y sus correspondientes cuerpos o ejemplares de relación).

Restricciones

Se define un **esquema de base de datos relacional** cómo un conjunto de esquemas de relaciones y otro conjunto de restricciones de integridad. Y se puede definir un **ejemplar de base de datos relacional** cómo un conjunto de ejemplares de relación con esquema de relación pertenecientes al esquema de base de datos relacional correspondiente, que a su vez satisfagan las restricciones de integridad pertenecientes al conjunto de restricciones de integridad perteneciente al esquema de base de datos relacional correspondiente.

- Restricciones de Integridad: Constantemente un DBMS relacional está verificando que no se violen las reglas de integridad definidas (por el usuario) en la definición del esquema de la base de datos.
 - *Restricciones de dominio*: Hemos mencionado que el esquema de cada relación define el tipo o dominio de cada atributo de cualquier tupla perteneciente al ejemplar de relación. Esto se traduce cómo una condición que cada tupla necesita satisfacer,

una restricción de dominio que sólo permite la existencia de valores de un dominio particular en un atributo de una tupla perteneciente al ejemplar de relación, por tanto se puede extender la definición de ejemplar de relación a la definición dada más la condición de que se satisfagan todas las restricciones de dominio.

- *Restricciones de clave:* Consiste en una declaración de que ciertos atributos de un esquema de relación pueden identificar unívocamente a las tuplas perteneciente al ejemplar de relación. Volviendo al ejemplo de la relación *Empleados*, podríamos decir que una **clave candidata** sería el {legajo}, ya que parece evidente que no puede haber números de legajos repetidos. Cabe destacar que una relación puede tener más de una clave candidata, podríamos agregar al esquema de relación de *Empleados* un par de atributos dni:integer y sexo:string, de modo que exista una clave candidata {dni, sexo}. Una clave candidata satisface las siguientes condiciones:
 - No pueden existir dos tuplas dentro de un ejemplar de relación que contengan valores idénticos en todos sus atributos pertenecientes a la clave.
 - No puede existir una clave compuesta por un subconjunto del conjunto de atributos de la clave (por ejemplo, no podría existir una clave candidata {sexo} en el ejemplo anterior.

Cualquier clave que contenga a una clave candidata y otros atributos, por ejemplo {legajo,nombre} se dice que es una superclave, que se define como un conjunto de campos o atributos que contiene una clave.

De este modo, queda garantizada la existencia de al menos una clave para toda relación, ya que de no poder distinguir ninguna, la clave se compone del conjunto de todos los atributos definidos en el esquema de relación.

Es común tomar todas las claves candidatas de una relación y designar una cómo la clave **principal** o **primaria**. De esta manera, se puede hacer referencia a cualquier tupla del ejemplar de relación vía la clave primaria. Por ejemplo, se puede distinguir una tupla de la relación *Empleados* vía la enunciación del legajo (ej.: legajo 131253).

- *Restricciones de integridad de entidades:* Consiste en una declaración por la cual ningún atributo (o campo) perteneciente a una clave primaria (de una relación determinada) puede ser nulo. Esto puede ser evidente, ya que si se admitiera en una clave el uso de un nulo en alguno de sus atributos, podríamos estar duplicando valores porque no podríamos determinar unívocamente la identidad de una tupla.
- *Restricciones de integridad referencial:* Es una restricción que sirve para establecer consistencia entre tuplas de dos relaciones diferentes. Nos da un valor agregado a nivel semántico en el modelo relacional. Supongamos que a la relación *Empleados*, le agregamos un atributo "país" para determinar el país de nacimiento del empleado en cuestión. Mal podríamos hablar de consistencia de nuestro modelo si permitiéramos el uso de países inexistentes ("La Luna", "República de La Boca"). Para evitar esto, definiremos una relación *Países* en dónde almacenaremos los países que consideramos válidos:

id_pais	nombre
1	Argentina
2	Chile
n	

Dónde {id_pais} es la clave primaria de la relación *Países*. Redefiniremos también la relación *Empleados* de la siguiente manera:

legajo	nombre	fecha_ingreso	id_pais
131250	Carlos Padilla	15/03/1978	1
131251	Ramiro Ochoa	17/10/2004	1
131252	Daniel Ramirez	10/04/2007	2
131253	Agustín Tarco	25/05/1999	1
131254	Javier Salazar	15/03/2007	1

Dónde, por ejemplo, el empleado con legajo 131252 es de nacionalidad chilena. Una restricción de integridad referencial nos debiera impedir establecer un valor del campo *id_pais* en la relación *Empleados* con un valor inexistente en la clave primaria de la relación *País*. Una vez presentado este modelo de ejemplo, conviene definir el concepto de **clave foránea** (foreign key). Podemos decir que un conjunto de atributos, por ejemplo $FK = \{id_pais\}$ de la relación *Empleados* es clave foránea de la relación *Empleados* si cumple con las siguientes condiciones:

- Los atributos FK hacen referencia a la relación referenciada (*Países*, por poner un ejemplo).
- Los atributos FK tienen el mismo dominio que los atributos de la clave primaria de la relación referenciada (*Países*, por poner un ejemplo).
- Un valor particular de los atributos FK en una tupla *t1* de la relación de referencia puede:
 - O bien ser nulo (si no hay una restricción especificada que indique lo contrario)
 - O existe una tupla *t2* en la relación referenciada (*Países*, por poner un ejemplo) tal que los atributos de la clave de *Países* contienen los

mismos valores para los atributos correspondientes en $t1$. Dicho más sencillo, pueden ocurrir (o existir) en la clave primaria de la relación referenciada.

- *Restricciones de integridad semántica:* Existen una variedad de restricciones que no corresponden a las antes descritas y tienen que ver con la consistencia que el diseñador de la base de datos desea imponer. Por ejemplo, en la relación Empleados, podríamos establecer que no deseamos tener empleados de países que no sean latinoamericanos. O que, si por ejemplo almacenáramos la edad de los empleados, quisiéramos tener empleados mayores a 18 años. Este tipo de restricciones que responden a un predicado lógico tienen diversas maneras de implementarse en los DBMS comerciales.

Operando con Restricciones

Veamos cómo puede comportarse un DBMS cuándo realiza las operaciones básicas en esquemas de base de datos que tienen definidas restricciones:

- Insertar:
 - Evaluar las restricciones de dominio. Desde el punto de vista de la programación, podemos analogarlo con un estilo de programación fuertemente tipado. En caso de encontrar alguna diferencia entre lo que se intenta insertar y el conjunto de dominio del atributo en particular, el DBMS arrojará un error y no permitirá la inserción de la tupla en la relación.
 - Evaluar las restricciones de integridad de entidades: Si se intenta insertar una tupla con un valor nulo en alguno de los atributos pertenecientes a la clave, también el DBMS impedirá la inserción.
 - Evaluar las restricciones de clave: Si la clave de la tupla a insertar ya existe entre las tuplas de la relación, el DBMS arrojará un error y no permitirá la inserción de la tupla en la relación.
 - Evaluar restricciones de integridad semántica: Si se viola alguna de las reglas de integridad establecidas en el esquema de base de datos, el DBMS impedirá la inserción de la tupla en la relación.
 - Evaluar restricciones de integridad referencial: Si se intenta insertar una tupla con un valor en un atributo perteneciente a una clave foránea que no se corresponda con ningún atributo perteneciente a la clave primaria de la relación referenciada, el DBMS impedirá la inserción de la tupla en la relación.
- Eliminar:

- Evaluar restricciones de integridad semántica: Hay que tener en cuenta que quizás la existencia de una tupla en una relación puede garantizar la satisfacción de una restricción de integridad semántica en otra relación (o quizás en la misma), de este modo, el DBMS impedirá la eliminación de la tupla, dado que dejaría la base de datos en un estado no íntegro.
- Evaluar restricciones de integridad referencial: Puede que cuando intentemos eliminar una tupla, exista una referencia a otra relación, en la cual exista una clave foránea, y una tupla que haga referencia a la que intentamos eliminar. En este caso el DBMS puede:
 - Rechazar la operación de eliminación (generalmente es la opción por defecto en los DBMS)
 - Eliminar las tuplas "huérfanas", es decir recorrer todo el árbol de integridad referencial necesario eliminando los registros que no tienen sentido que permanezcan "huérfanos" (en este caso, se establece en la definición de las claves foráneas, que ante una eliminación, se actúe en forma de cascada).
 - Asignarle a las tuplas referenciadas un valor predeterminado para que la restricción se siga satisfaciendo (en este caso, se establece en la definición de las claves foráneas, que ante una eliminación, se tome un valor único como predeterminado o default)
 - En forma similar al punto anterior, asignar un null si está permitido en la relación referenciada.
- Modificar:
 - Evaluar restricciones de dominio: De la misma manera que en la operación insertar, se pueden introducir problemas de tipado. De este modo, cuando ocurre un problema de restricción de dominio, el DBMS no permite la actualización de las tuplas en cuestión.
 - Evaluar las restricciones de integridad de entidades: Si se intenta actualizar una tupla con un valor nulo en alguno de los atributos pertenecientes a la clave, también el DBMS impedirá la actualización.
 - Evaluar las restricciones de clave: Si la clave de la tupla a actualizar ya existe entre las tuplas de la relación, el DBMS arrojará un error y no permitirá la actualización de la tupla en la relación. Cabe destacar que si bien el modelo permite actualizar atributos de relaciones que estén en el conjunto de atributos de la clave primaria, muchas veces puede evidenciar un problema de diseño, ya que si algo es realmente un identificador unívoco de una entidad, no parece razonable que dicho atributo pueda mutar en el tiempo. Por ejemplo no parece razonable que un empleado cambie de legajo.

- Evaluar restricciones de integridad semántica: Si se viola alguna de las reglas de integridad establecidas en el esquema de base de datos, el DBMS impedirá la actualización de la tupla en la relación.
- Evaluar restricciones de integridad referencial: Puede que cuando intentemos actualizar una tupla, exista una referencia a otra relación, en la cual exista una clave foránea, y una tupla que haga referencia a la que intentamos actualizar y podamos estar dejando registros sin referencia. En este caso el DBMS puede:
 - Rechazar la operación de actualización (generalmente es la opción por defecto en los DBMS)
 - Actualizar las tuplas "huérfanas", es decir recorrer todo el árbol de integridad referencial necesario actualizando las claves de los registros que no tienen sentido que permanezcan "huérfanos" (en este caso, se establece en la definición de las claves foráneas, que ante una actualización, se actúe en forma de cascada).
 - Asignarle a las tuplas referenciadas un valor predeterminado para que la restricción se siga satisfaciendo (en este caso, se establece en la definición de las claves foráneas, que ante una actualización de una clave en la relación de referencia, se tome un valor único como predeterminado o default)
 - En forma similar al punto anterior, asignar un null si está permitido en la relación referenciada.

El Modelo Entidad-Relación

En 1976, Chen introdujo este modelo como una manera de modelar problemáticas reales desde el punto de vista del almacenamiento de datos. Fue también objetivo de Chen proveer un marco unificador para el modelado de los datos. Durante mucho tiempo este modelo ha sido el estándar único para modelar bases de datos relacionales y en realidad provee una visión unificadora de los modelos jerárquicos, de red y relacional; es decir, una generalización y extensión de los modelos citados.

Establece que durante el estudio de un modelo de datos, surgen abstracciones de entidades, que consisten en "cosas" que pueden ser directamente identificadas. Por ejemplo durante el análisis de una problemática de facturación, una entidad que surgirá seguramente será la "factura". Otra abstracción que surge es la de "relación" (que es un concepto diferente al de "relación" en el modelo relacional) que consiste en una asociación entre entidades. Por ejemplo la relación "cabecera-detalle" dentro de una factura es una relación entre las cabeceras de las entidades facturas y los detalles de las entidades "*detalle de factura*". El rol de una entidad cuando se abstrae una relación consiste en la

función que realiza, por ejemplo "matrimonio" es una relación entre dos entidades "*persona*", pero "*esposo*" y "*esposa*" son roles de las entidades dentro de la relación "*matrimonio*".

La información contenida en una entidad se obtiene por medición u observación y se almacena en pares (atributo, valor). El concepto de conjunto de valores es análogo al de dominio o tipo en el modelo relacional.

Las relaciones también poseen atributos, por ejemplo si existe la entidad "*Empleado*" y la entidad "*Proyecto*", puede que muchos empleados trabajen en muchos proyectos diferentes, y si quisiera almacenar la cantidad de tiempo que trabaja cada empleado en cada proyecto, probablemente pueda modelar el atributo "horas trabajadas" como perteneciente a la relación "*Empleado-Proyecto*", ya que no tiene sentido asociarlo como atributo de la entidad empleado (ya que puede trabajar en varios proyectos) y tampoco tiene sentido asociarlo a la entidad proyecto (ya que puede tener muchos empleados).

El concepto de clave primaria para las entidades es análogo al de las relaciones del modelo relacional.

Diagramas de Entidad-Relación

Una forma sencilla de representar un modelo de Entidad Relación es mediante esta herramienta. Las convenciones para la representación varían de autor en autor y podemos considerarlas a todas válidas, por lo que la elección que tomamos puede ser arbitraria. Generalmente un DER (Diagrama Entidad-Relación) no se puebla con atributos ni con claves. Las entidades se representan en un diagrama como rectángulos en donde se inscribe su nombre, salvo en las siguientes excepciones:

- Entidades intermedias: Sirven para representar una relación muchos a muchos que tiene semántica en sí misma (como el ejemplo de la relación Empleado-Proyecto enunciado anteriormente. Se diagrama como un rombo.
- Entidades débiles: Son entidades que no tienen razón de ser sino por otra entidad. En el análisis orientado a objetos se puede corresponder con las clases componentes. Generalmente se las diagrama como un rectángulo con un borde más fuerte.

Las relaciones se diagraman como flechas o líneas que unen entidades:

- Relaciones one-to-one: Establece una relación entre entidades en la cual una entidad de origen se relaciona con una entidad de destino. Por ejemplo:



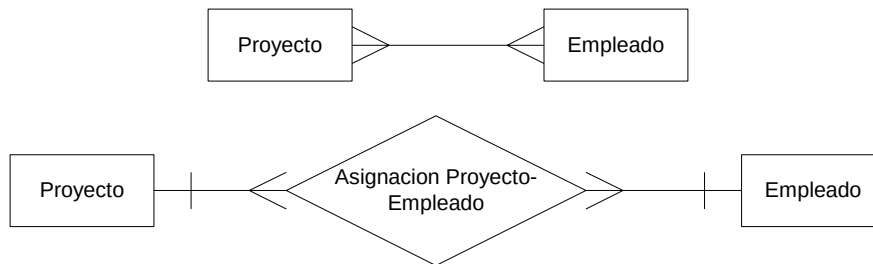
- Relaciones one-to-many: Establece una relación entre entidades en la cual una entidad de origen se relaciona con muchas entidades de destino. Por ejemplo:



- Relaciones many-to-one: Es la visión inversa del caso anterior. Ejemplo:

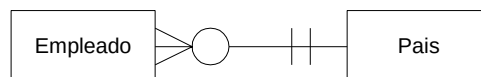


- Relaciones many-to-many: Puede verse cómo la composición de una relación one-to-many y una many-to-one, o cómo una relación en la cual una entidad de cualquiera de los dos conjuntos de entidades relacionadas puede estar relacionado con muchas entidades del otro conjunto de entidades. Ejemplos:



Del modo presentado, vemos cómo la representación de las relaciones muestra la cardinalidad de las mismas, es decir, queda claramente diagramado que un país se relaciona con muchos empleados, pero que un empleado sólo se relaciona con un país. Para completar el diagrama, podríamos agregar la obligatoriedad de la relación. Por ejemplo ¿Un empleado tiene que tener una relación con un país? ¿O puede no tener ninguna? ¿Un país, para ser considerado tal, tiene que tener al menos un empleado relacionado? ¿O no?

Supongamos que definimos que un empleado necesariamente tiene que tener relación con un país, pero que un país puede existir en nuestra tabla sin necesidad de tener un empleado relacionado, podríamos diagramarlo de la siguiente manera:



Ejemplo de Modelado Entidad-Relación

Supongamos el siguiente enunciado:

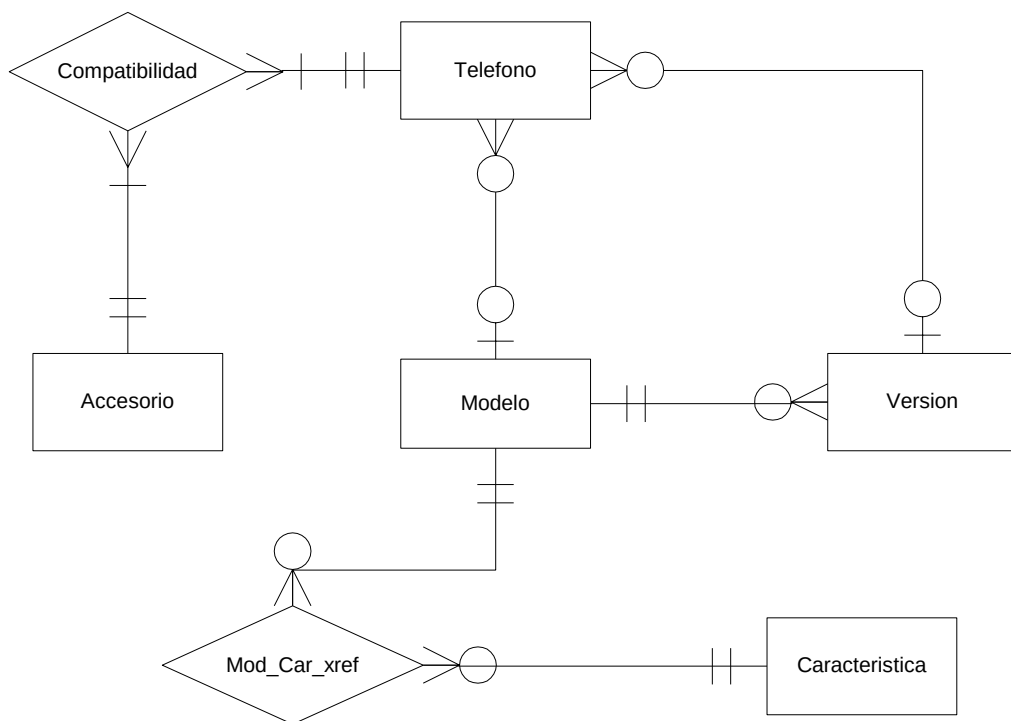
Una empresa de teléfonos móviles quiere construir una base de datos de los productos que vende. La empresa fabrica varios teléfonos y accesorios. Los teléfonos tienen modelos y para cada modelo hay varias versiones (ejemplo, X-180 significa modelo X, versión 190). Cada modelo tiene ciertas características (WAP, Bluetooth, Juegos, tonos polifónicos, etc.), y las distintas versiones se diferencian solo en que tienen distinto cierto tamaño y peso. Además la empresa vende accesorios (auriculares, cargadores, cámaras, etc.) que son compatibles con varios de sus teléfonos, pero no con todos.

Además de saber qué teléfonos y accesorios fabrica y las características y compatibilidad de cada uno, la empresa quiere saber exactamente cuántas unidades de cada cosa tiene (si guardar información repetida) cuánto cuesta fabricar cada ítem que fabrica y en qué precio vende cada ítem que fabrica.

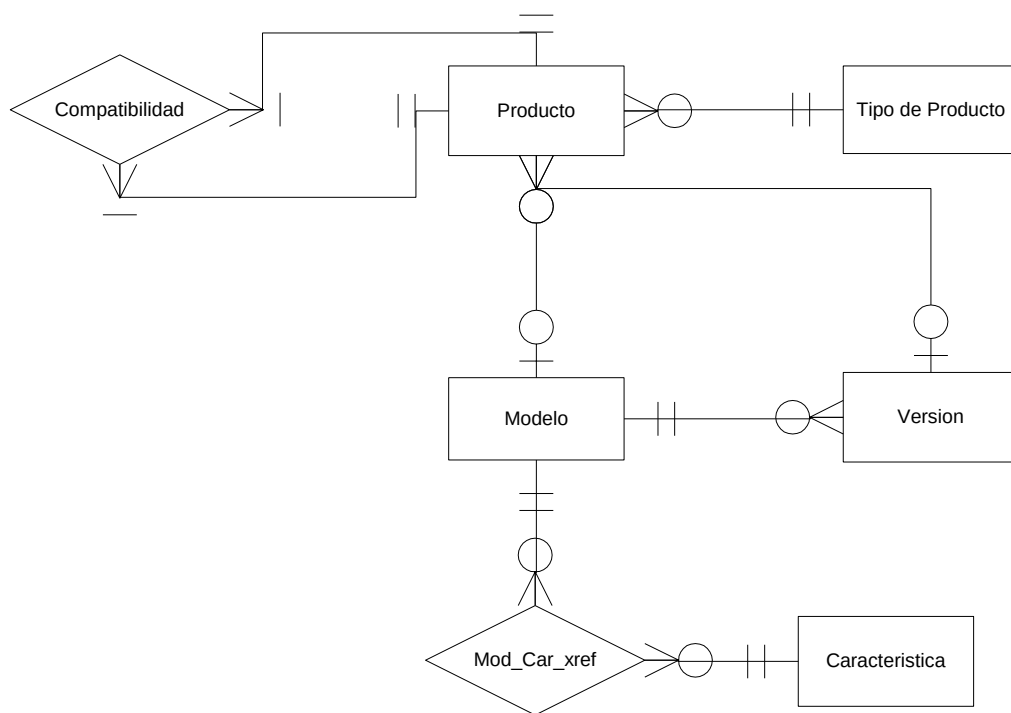
Podríamos modelarlo de la siguiente manera tomando como base las siguientes afirmaciones:

- Utilizaremos singulares para nomenciar las entidades
- No utilizaremos acentos del idioma español para nomenciar entidades
- Tomaremos dos vías distintas de diseño:
 - Agruparemos los teléfonos y accesorios en una entidad más fuerte llamada Producto, y determinaremos su pertenencia a Accesorios o Teléfonos (o un futuro rubro) mediante la entidad Tipo de Producto.
 - Existirán entidades autónomas Teléfono y Accesorio.

El primer modelo:



El segundo modelo:



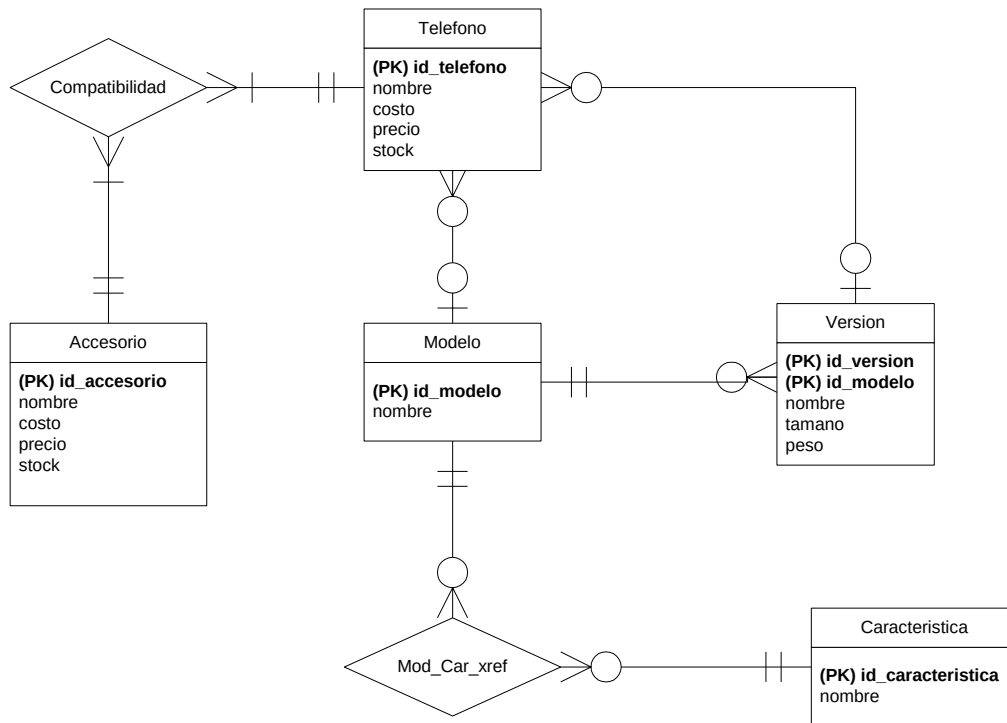
Las diferencias entre la utilización de uno de los modelos u otro no residen en mayor correctitud o no, sino en conceptos de análisis, tales como escalabilidad, ahorro de almacenamiento, performance, elegancia de diseño.

Del Modelo ER al Modelo Relacional

Generalmente, una vez concebido un diagrama de Entidad-Relación, se puede transformar el análisis hecho en uno o varios esquemas relacionales, de hecho, consistiría en un paso previo al diseño físico de la base de datos.

Cabe, antes que nada, establecer una analogía entre componentes de modelo entidad relación y del modelo relacional, aunque puede que la misma resulte trivial. Cuando en ER definimos una entidad, el concepto correspondiente del lado del modelo relacional, es la relación (o mejor, el esquema de relación). Asimismo, en el modelo relacional podemos definir claves primarias, cuyo concepto análogo en ER es el de definir un atributo (cuando poblamos el DER) clave que identificará unívocamente la entidad a la que pertenece. Con respecto a las relaciones en el ER, podríamos dividirla en dos tipos, las relaciones one-to-many y las many-to-many (teniendo en cuenta que la relación many-to-one, no es más que una visión invertida de la relación one-to-many). La relación one-to-many del modelo ER se corresponde con el concepto de clave foránea (o foreign key) del modelo relacional. Con respecto a la relación many-to-many, no sólo está emparentada con las claves foráneas, sino que la entidad intermedia (o de cruce) es también una relación en el modelo relacional. En el modelo ER existe la posibilidad de definir atributos multivaluados (fechas de visita, etc.), que se corresponden en el modelo relacional con la existencia de otra relación y de otra clave foránea, este concepto será explotado cuando se estudie "*Normalización*".

A modo de ejemplo, transformaremos el primer ejemplo dado de un modelo ER de teléfonos celulares, en un esquema relacional. En primer lugar, poblaremos de atributos y claves el modelo ER dado:



Las relaciones many-to-many no se han poblado, ya que en este caso no poseen atributos particulares.

En primer lugar, vamos a definir un esquema de relación (modelo relacional) por cada entidad definida en el modelo ER, asignándole la clave (PK) que corresponda en el modelo ER y todos los atributos (no repetitivos) de la misma:

- Teléfono (**id_telefono:integer** ; nombre:string ; costo:double ; precio:double ; stock:integer)
- Accesorio (**id_accesorio:integer** ; nombre:string ; costo:double ; precio:double ; stock:integer)
- Modelo (**id_modelo:integer** ; nombre:string)
- Versión (**id_version:integer** ; **id_modelo:integer** ; nombre:string ; tamano:double ; peso:double)
- Característica (**id_caracteristica:integer** ; nombre:string)
- Compatibilidad ()
- Mod_Car_xref ()

Ahora, por cada relación one-to-one existen dos posibilidades:

- Crear una nueva relación, con su clave primaria y definir en la relación que corresponda la clave foránea que mapee ambas relaciones.
- Juntar ambas entidades en una misma relación. Esto es conveniente cuando la obligatoriedad es mandatoria, de modo que siempre existe una tupla que se relaciona en ambas relaciones.

Cómo en nuestro caso no hay relaciones one-to-one, seguiremos adelante. Por cada relación one-to-many, se agregarán todos los atributos de la clave primaria de la relación del lado "one" en la relación del lado "many" y se los definirá en una nueva clave foránea. Por convención señalaremos los atributos de clave primaria con letra **negrita** y los atributos de clave foránea, con subrayado. El esquema relacional, quedaría de la siguiente manera:

- Teléfono (**id_telefono:integer** ; nombre:string ; costo:double ; precio:double ; stock:integer)
- Accesorio (**id_accesorio:integer** ; nombre:string ; costo:double ; precio:double ; stock:integer)
- Modelo (**id_modelo:integer** ; nombre:string)
- Versión (**id_version:integer** ; id_modelo:integer ; nombre:string ; tamano:double ; peso:double)
- Característica (**id_caracteristica:integer** ; nombre:string)
- Compatibilidad (id_telefono:integer ; id_accesorio:integer)
- Mod_Car_xref (id_caracteristica:integer ; id_modelo:integer)

De esta forma, quedaron incluidas las relaciones many-to-many ya que fueron vista cómo la composición de dos relaciones one-to-many. Si en la representación del modelo ER, no estuvieran las relaciones many-to-many representadas con una entidad intermedia, durante la transformación al modelo relacional, debería crearse una relación que represente la entidad intermedia.

Si existieran atributos multivaluados, se crearía una relación nueva para representar dicho atributo, de la misma manera cómo se representa una relación one-to-many.

Bibliografía y referencias

- "Sistemas de Bases de Datos - Conceptos fundamentales" - Elmasti/Navathe - Segunda Edición Addison Wesley Iberoamericana.
- "Sistemas de Gestión de Bases de Datos" - Ramakrishnan/Gerke - Tercera edición Ed. McGraw Hill
- "Introducción a los sistemas de Bases de Datos" - C.J. Date - Séptima edición Ed. McGraw Hill
- Wikipedia (<http://wikipedia.org>)